Documentación – Parser CYK en Python

1. Investigación

El algoritmo CYK (Cocke-Younger-Kasami) decide si una cadena w pertenece al lenguaje de una gramática libre de contexto. Usa programación dinámica y exige que la gramática esté en Forma Normal de Chomsky (FNC).

Forma Normal de Chomsky

Una gramática está en FNC cuando todas sus producciones tienen una de estas formas:

- A -> B C, donde B y C son no terminales.
- A -> a, donde a es un terminal.
- S -> ε, solo para el símbolo inicial y solo si S no aparece en ningún lado derecho.

Toda gramática libre de contexto se puede convertir a FNC sin cambiar su lenguaje. Para eso primero se simplifica (se eliminan producciones épsilon, producciones unitarias y símbolos inútiles) y luego se ajustan los cuerpos para que tengan exactamente dos no terminales o un terminal.

Programación dinámica

La programación dinámica resuelve un problema grande combinando soluciones de subproblemas más pequeños que se guardan en una tabla para no recalcularlos. En CYK el subproblema es: qué no terminales generan la subcadena que va de la palabra i a la palabra j.

2. Diseño de la aplicación

La aplicación está escrita en Python 3, sin librerías externas, con un diseño modular: cada etapa vive en su propio módulo y cada operación es una función separada. Las funciones son puras: reciben una gramática y devuelven una nueva sin modificar la original.

Estructura de archivos

- main.py: programa principal (línea de comandos y modo interactivo).
- ejemplos.py: ejecuta las frases de ejemplo dentro y fuera del lenguaje.
- pruebas.py: 21 pruebas automáticas con unittest.
- gramaticas/gramatica_ingles.txt: la gramática del enunciado.
- gramaticas/gramatica_simplificacion.txt: gramática de prueba con ε, unitarias y símbolos inútiles.
- modulos/gramatica.py: estructura de datos, lectura y presentación de la gramática.
- modulos/simplificacion.py: algoritmo de simplificación.
- modulos/fnc.py: conversión a Forma Normal de Chomsky.
- modulos/cyk.py: algoritmo CYK.
- modulos/arbol.py: árbol de derivación.
- modulos/utilidades.py: lectura de la frase, medición de tiempo y tabla en texto.

Representación de los datos

- Gramática: clase Gramatica con tres campos. inicial es el símbolo inicial. producciones es un diccionario {no terminal: conjunto de tuplas}; por ejemplo S -> NP VP se guarda como "S": {("NP", "VP")}. auxiliares guarda los no terminales creados en la conversión a FNC, para poder mostrar el árbol con la gramática original.
- Épsilon: la tupla vacía ().
- Terminales y no terminales: un símbolo es no terminal si empieza con mayúscula (S, NP, Det) y terminal en otro caso (cooks, the).
- Tabla CYK: matriz n x n; cada celda es un diccionario {no terminal: lista de referencias}. Una referencia es ("hoja", palabra) o ("rama", k, B, C).
- Árbol: clase Nodo con etiqueta y lista de hijos.

Funciones por módulo

modulos/gramatica.py

- es_no_terminal, es_terminal: clasifican un símbolo.
- agregar_produccion: agrega una producción al diccionario.
- separar_lados, interpretar_alternativa, interpretar_linea: leen una línea del archivo.
- leer_gramatica_desde_texto, leer_gramatica_desde_archivo: construyen la Gramatica.
- copiar_gramatica, obtener_no_terminales, obtener_terminales, contar_producciones.
- generar_nombre_unico: crea nombres nuevos sin choques (S0, T_a, S_1).
- ordenar_no_terminales, cuerpo_a_texto, gramatica_a_texto: presentación.

modulos/simplificacion.py

- calcular_anulables, generar_variantes_sin_anulables, eliminar_producciones_epsilon.
- es_produccion_unitaria, calcular_pares_unitarios, eliminar_producciones_unitarias.
- calcular_simbolos_generadores, eliminar_simbolos_no_generadores.
- calcular_simbolos_alcanzables, eliminar_simbolos_no_alcanzables.
- eliminar_simbolos_inutiles: no generadores y luego no alcanzables.
- simplificar_gramatica: épsilon, unitarias e inútiles en ese orden.

modulos/fnc.py

- inicial_aparece_a_la_derecha, agregar_nuevo_simbolo_inicial.
- reemplazar_terminales_en_cuerpos_largos, binarizar_producciones.
- obtener_violaciones_fnc, es_forma_normal_chomsky: verifican el resultado.
- convertir_a_fnc: ejecuta los seis pasos y devuelve la gramática de cada paso.

modulos/cyk.py

- construir_indice_lexico: {terminal: [A]} para A -> a.
- construir_indice_binario: {(B, C): [A]} para A -> B C.
- crear_tabla_vacia, llenar_diagonal, llenar_celda, llenar_tabla.
- acepta_cadena_vacia, algoritmo_cyk: devuelve (pertenece, tabla).

modulos/arbol.py

- construir_arbol: reconstruye el árbol desde las referencias de la tabla.
- contar_arboles: cuenta las derivaciones distintas (detecta ambigüedad).
- revertir_nodo, restaurar_arbol_original: quitan los símbolos auxiliares de la FNC.
- obtener_hojas, arbol_a_corchetes, arbol_a_lineas, arbol_a_texto: presentación.

modulos/utilidades.py

- normalizar_frase, tokenizar_frase: minúsculas, sin puntuación, lista de palabras.
- medir_tiempo, formatear_tiempo: tiempo con time.perf_counter en milisegundos.
- celda_a_texto, tabla_cyk_a_texto: tabla CYK en texto.

Flujo del programa

1. Se lee la gramática desde el archivo de texto.
2. Se simplifica y se convierte a FNC; se verifica que el resultado cumpla la FNC.
3. Se lee la frase w y se divide en palabras.
4. Se ejecuta CYK y se mide su tiempo.
5. Se imprime SÍ o NO y el tiempo.
6. Si la respuesta es SÍ, se construye e imprime el árbol de derivación.

3. Simplificación y conversión a FNC

La conversión aplica seis pasos en orden. Los pasos 2 a 4 son el algoritmo de simplificación y los pasos 5 y 6 completan la FNC.

1. Nuevo símbolo inicial: si S aparece en algún lado derecho se crea S0 -> S. Así S0 -> ε nunca queda dentro de otro cuerpo.
2. Producciones épsilon: se calculan los anulables con un punto fijo. Cada producción se reemplaza por todas sus variantes quitando o dejando cada anulable, y se descartan las A -> ε (solo el inicial conserva ε si era anulable).
3. Producciones unitarias: se calculan los pares (A, B) con A =>* B por unitarias. A recibe las producciones no unitarias de B y se descartan las unitarias.
4. Símbolos inútiles: primero se quitan los no generadores y luego los no alcanzables desde el inicial. En el orden contrario podrían quedar símbolos inútiles.
5. Terminales en cuerpos largos: en cuerpos de 2 o más símbolos cada terminal t se cambia por T_t, con T_t -> t.
6. Binarización: A -> X1 X2 ... Xn se cambia por A -> X1 A_1, A_1 -> X2 A_2, ..., A_(n-2) -> X(n-1) Xn.

Al final se verifica con obtener_violaciones_fnc que todas las producciones cumplan la FNC.

Gramática del enunciado

La gramática del enunciado ya está en FNC: no tiene producciones épsilon ni unitarias, S no aparece a la derecha, todos sus símbolos son útiles y cada cuerpo es de dos no terminales o un terminal. El algoritmo la recorre completa y la devuelve con las mismas 30 producciones:

```
S -> NP VP
VP -> V NP | VP PP | cooks | cuts | drinks | eats
PP -> P NP
```

```
NP -> Det N | he | she
V -> cooks | cuts | drinks | eats
P -> in | with
N -> beer | cake | cat | dog | fork | juice | knife | meat | oven | soup | spoon
Det -> a | the
```

Gramática de prueba de la simplificación

Como la gramática del enunciado no ejercita los pasos, se usó una segunda gramática (gramaticas/gramatica_simplificacion.txt) con producciones ε, unitarias, un no generador (D), un no alcanzable (E), terminales en cuerpos largos y un cuerpo de tres símbolos. Su lenguaje es a* seguido de una b opcional seguido de c*.

Original (11 producciones):

```
S -> A B C | D | a S
A -> ε | a A
B -> A | b
C -> ε | c C
D -> d D
E -> e
```

Paso 1, nuevo símbolo inicial (12 producciones): se agrega S0 -> S porque S aparece en a S.

Paso 2, sin épsilon (20 producciones). Anulables: S, A, B, C.

```
S0 -> ε | S
S -> A | A B | A B C | A C | B | B C | C | D | a | a S
A -> a | a A
B -> A | b
C -> c | c C
D -> d D
E -> e
```

Paso 3, sin unitarias (32 producciones):

```
S0 -> ε | A B | A B C | A C | B C | a | a A | a S | b | c | c C | d D
S -> A B | A B C | A C | B C | a | a A | a S | b | c | c C | d D
A -> a | a A
B -> a | a A | b
C -> c | c C
D -> d D
E -> e
```

Paso 4, sin símbolos inútiles (28 producciones): se eliminan D (no generador) y E (no alcanzable).

Pasos 5 y 6, FNC final (32 producciones):

```
S0 -> ε | A B | A C | A S0_1 | B C | T_a A | T_a S | T_c C | a | b | c
S -> A B | A C | A S_1 | B C | T_a A | T_a S | T_c C | a | b | c
A -> T_a A | a
B -> T_a A | a | b
C -> T_c C | c
T_a -> a
T_c -> c
S0_1 -> B C
S_1 -> B C
```

Árbol de derivación

Cada celda no guarda solo un valor booleano: para cada no terminal guarda referencias de cómo se obtuvo. ("hoja", palabra) indica A -> palabra; ("rama", k, B, C) indica A -> B C con el corte en k. construir_arbol parte de T[0][n-1] con el símbolo inicial y sigue las referencias recursivamente hasta las hojas. contar_arboles usa las mismas referencias, con memoización, para contar cuántas derivaciones distintas existen; si hay más de una la frase es ambigua.

Si la conversión creó símbolos auxiliares (S0, T_a, S_1), restaurar_arbol_original los quita: T_a se reemplaza por la hoja a, los nodos binarios auxiliares pasan sus hijos al padre y S0 se renombra como S. El programa imprime el árbol en texto con sangría y en notación de corchetes.

Medición del tiempo

medir_tiempo usa time.perf_counter, el reloj de mayor resolución de Python. Se reportan por separado el tiempo de simplificación y conversión a FNC (se hace una sola vez al cargar la gramática), el tiempo del algoritmo CYK y el tiempo de construcción del árbol.

4. Uso del programa

Requiere Python 3.8 o superior. Se ejecuta desde la carpeta proyecto_cyk.

```
python main.py "She eats a cake with a fork"
python main.py                      (modo interactivo, termina con 'salir')
python main.py --tabla "The cat drinks the beer"
python main.py --pasos --gramatica gramaticas/gramatica_simplificacion.txt "a a b c"
python ejemplos.py
python pruebas.py
```

- --tabla: muestra la tabla dinámica de CYK.
- --pasos: muestra la gramática después de cada paso de la conversión a FNC.
- --gramatica RUTA: usa otra gramática.

La frase se pasa a minúsculas y se le quitan los signos de puntuación, así que "She eats a cake with a fork." y "she eats a cake with a fork" son la misma entrada.

Formato del archivo de gramática: una producción por línea con -> o →, alternativas separadas con |, ε (o epsilon) para la producción vacía y # para comentarios. El primer lado izquierdo es el símbolo inicial.

Salida para w = She eats a cake with a fork (con --tabla se agrega la tabla):

```
Frase: She eats a cake with a fork
Palabras: ['she', 'eats', 'a', 'cake', 'with', 'a', 'fork']
Resultado: SÍ
Tiempo del algoritmo CYK: 0.0517 ms

Árboles de derivación posibles: 1
Tiempo de construcción del árbol: 0.0254 ms

Árbol de derivación (gramática en FNC):
S
|-- NP
|   `-- she
```

```
`-- VP
    |-- VP
    |   |-- V
    |   |   `-- eats
    |   `-- NP
    |       |-- Det
    |       |   `-- a
    |       `-- N
    |           `-- cake
    `-- PP
        |-- P
        |   `-- with
        `-- NP
            |-- Det
            |   `-- a
            `-- N
                `-- fork

Notación con corchetes:
[S [NP she] [VP [VP [V eats] [NP [Det a] [N cake]]] [PP [P with] [NP [Det a] [N
fork]]]]]
```

Celdas no vacías de la tabla CYK para esa frase:

```
[0,0] she    {NP}        [0,1] she eats                    {S}
[1,1] eats   {V, VP}     [2,3] a cake                      {NP}
[2,2] a      {Det}       [5,6] a fork                      {NP}
[3,3] cake   {N}         [1,3] eats a cake                 {VP}
[4,4] with   {P}         [4,6] with a fork                 {PP}
[5,5] a      {Det}       [0,3] she eats a cake             {S}
[6,6] fork   {N}         [1,6] eats a cake with a fork     {VP}
                         [0,6] she eats a cake with a fork {S}
```

5. Ejemplos y pruebas realizadas

Las 9 frases del lenguaje se aceptaron y las 10 frases fuera del lenguaje se rechazaron. Los tiempos son de una ejecución de ejemplos.py en Python 3.13; cambian ligeramente entre ejecuciones y equipos.

Frases que pertenecen al lenguaje

Frase	Resultado	Tiempo CYK
She eats a cake with a fork	SÍ	0.0517 ms
The cat drinks the beer	SÍ	0.0304 ms
He cooks	SÍ	0.0166 ms
she eats	SÍ	0.0163 ms
The dog eats the meat in the oven	SÍ	0.0363 ms
He cuts the cake with a knife	SÍ	0.0301 ms
She drinks the juice with a spoon in the oven	SÍ	0.0465 ms

Frase	Resultado	Tiempo CYK
A cat cooks a dog	SÍ	0.0223 ms
The fork eats the soup	SÍ	0.0223 ms

Árboles obtenidos (notación con corchetes):

```
The cat drinks the beer
[S [NP [Det the] [N cat]] [VP [V drinks] [NP [Det the] [N beer]]]]

He cooks
[S [NP he] [VP cooks]]

The dog eats the meat in the oven
[S [NP [Det the] [N dog]] [VP [VP [V eats] [NP [Det the] [N meat]]] [PP [P in] [NP
[Det the] [N oven]]]]]

She drinks the juice with a spoon in the oven
[S [NP she] [VP [VP [VP [V drinks] [NP [Det the] [N juice]]] [PP [P with] [NP [Det a]
[N spoon]]]] [PP [P in] [NP [Det the] [N oven]]]]]
```

The fork eats the soup y A cat cooks a dog se aceptan aunque no tengan sentido: CYK solo revisa la sintaxis, no la semántica.

Frases que no pertenecen al lenguaje

Frase	Resultado	Tiempo CYK	Motivo
Eats she	NO	0.0145 ms	El verbo va antes del sujeto
eats a cake	NO	0.0180 ms	Falta el sujeto NP
She cake	NO	0.0164 ms	No hay verbo
The cat	NO	0.0166 ms	Solo es un NP, falta VP
She eats cake	NO	0.0196 ms	N sin determinante no es NP
Cat drinks the beer	NO	0.0225 ms	Sujeto sin determinante
She eats a cake with	NO	0.0247 ms	PP incompleto
He drinks the juice quickly	NO	0.0243 ms	quickly no está en la gramática
she she eats	NO	0.0422 ms	Dos sujetos
The cat the dog eats	NO	0.0253 ms	Dos NP seguidos

Pruebas automáticas

pruebas.py contiene 21 pruebas con unittest y todas pasan (Ran 21 tests, OK). Cubren:

- Lectura: archivo, comentarios, ε y líneas inválidas.

- Simplificación: anulables {S, A, B, C}; sin ε fuera del inicial; pares unitarios; sin producciones unitarias; D no generador y E no alcanzable eliminados.
- FNC: S0 se crea solo cuando hace falta; T_a y T_b en S -> a S b; binarización de un cuerpo de 4 símbolos; ambas gramáticas quedan en FNC.
- Conservación del lenguaje: la gramática de prueba convertida acepta "", a, b, c, aabcc, ac, bc, aaa y rechaza ba, cb, bb, d, e, abca.
- CYK: frases aceptadas y rechazadas, incluida la frase vacía.
- Árbol: sus hojas reproducen la frase; con S -> a S b el árbol restaurado es S -> a S b sin auxiliares.
- Ambigüedad: al agregar NP -> NP PP, She eats a cake with a fork tiene 2 árboles (el tenedor puede modificar al verbo o al pastel).

8. Discusión

Obstáculos encontrados

- La gramática del enunciado ya está en FNC, así que no sirve para probar la simplificación. Se creó una segunda gramática que obliga a usar todos los pasos y se verificó que su lenguaje no cambia.
- Distinguir terminales de no terminales al leer el archivo. Se adoptó la convención de mayúscula inicial para no terminales; una palabra como "I" tendría que escribirse en minúscula en la gramática.
- El orden de los pasos importa. Las unitarias se eliminan después de ε porque quitar ε puede crear nuevas unitarias (S -> A B C produjo S -> A). Los no generadores se eliminan antes que los no alcanzables.
- Si S aparece a la derecha y además es anulable, S -> ε no se puede dejar tal cual; por eso se crea S0 al inicio.
- Los nombres nuevos (S0, T_a, S_1) pueden chocar con símbolos existentes; generar_nombre_unico agrega un sufijo cuando hace falta.
- La FNC cambia la forma del árbol. Se guardaron los símbolos auxiliares para deshacer T_a, S_1 y S0 y mostrar el árbol con la gramática original. La eliminación de unitarias no se puede revertir así: si existiera NP -> Name y Name -> john, el árbol mostraría NP -> john.
- La entrada trae mayúsculas y puntuación ("She", "fork."). Se normaliza la frase antes de dividirla en palabras.
- Los tiempos son de centésimas de milisegundo, así que se usó time.perf_counter en lugar de time.time, que no tiene resolución suficiente.

Recomendaciones

- Guardar referencias en la tabla en lugar de booleanos: cuesta poco y permite construir el árbol y detectar ambigüedad.
- Indexar las producciones por cuerpo ({(B, C): [A]}) para que cada celda no recorra toda la gramática.
- Separar cada transformación en una función pura y probarla por separado; así es fácil encontrar en qué paso falla la conversión.
- Verificar siempre al final que la gramática cumple la FNC antes de ejecutar CYK.

- Para frases largas o gramáticas grandes, CYK crece como $O(n^3 \cdot |G|)$; en esos casos conviene un parser de Earley o uno probabilístico.
- Ampliar el vocabulario (adjetivos, plurales, NP -> NP PP) para cubrir más frases, sabiendo que NP -> NP PP vuelve ambigua la gramática.